

TD7: Ingeniería de Datos

Big Data

Recorrido inicial

Big Data



Recorrido Big Data: pensando un sistema de datos escalable, ejemplo de *web analytics*:

- Recibe eventos de actividad de usuario
- Orientado a entender el tráfico de una página web
- Procesa los datos a medida que llegan
- Permite hacer consultas aleatorias

Big Data - SuperWebAnalytics.com



1 - Estado inicial:

➤ Base de datos relacional con **pageviews** inicial

user_id	url	date	pageviews
731	/home	14/10/2024	82
1711	/home	14/10/2024	11
1682	/home/careers	14/10/2024	2
241	/subscription_form	14/10/2024	1

Ante cada evento de pageview, se actualiza el registro correspondiente.

Big Data - SuperWebAnalytics.com



2 - Aumenta el tráfico:

➤ ¿Qué pasa al **escalar el tráfico**?

- ¿Qué **elementos** de una RDBMS pueden **ser un cuello de botella** y **producir timeouts**?

“Timeout error on inserting to the database.”

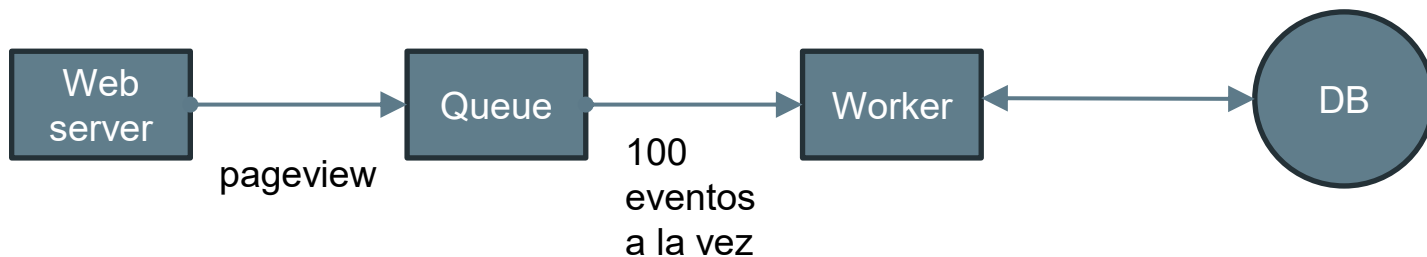
La **base de datos no puede mantener el ritmo de la carga**, por lo que las **solicitudes de escritura para incrementar las pageviews dan timeout**.

Big Data - SuperWebAnalytics.com



2 - Aumenta el tráfico:

- ¿Cómo podemos desacoplar los componentes?
- Acumular eventos para reducir la cantidad de llamadas a la base
- Worker independiente recibe los eventos y los almacena en una cola para manejar la back-pressure



Big Data - SuperWebAnalytics.com



— — —

3 - El tráfico aumenta todavía más:

- El worker empieza a atrasarse y la cola sólo crece
 - Se paraleliza esta tarea agregando más workers
 - Un load balancer distribuye la carga equitativamente

Big Data - SuperWebAnalytics.com



4 - La base de datos vuelve a ser cuello de botella:

➤ Es necesario **escalar la DB** también:

- **Multiplicar los servidores de DBs** y **desparramar la tabla en todos los servidores.**
- Cada **servidor** tendrá **un subconjunto de los datos de la tabla.**
 - Esta técnica se conoce como **partición horizontal o sharding:** **desparramar los write load en múltiples máquinas.**
- Para **elegir el shard al que va a parar cada clave** hay que **tomar el hash de la clave módulo el número de shards.**
- **Mapear las claves a los shards** se hace para que **estén uniformemente distribuidas en los shards.**

Big Data - SuperWebAnalytics.com



4 - La base de datos vuelve a ser cuello de botella:

- La aplicación necesita saber cómo encontrar el shard para cada clave:
 - Lee el número de shards de un archivo de configuración y calcula para cada clave el shard correcto.
 - Las queries se resuelven consultando todos los nodos y haciendo merge de los resultados:
 - Para obtener las 100 URLs más visitadas tiene que obtener las 100 URLs top de cada shard y hacer un merge para obtener el top global.
 - Si la aplicación se vuelve más popular hay que **resharding** la DB en más shards.
 - Es necesario hacer **resharding** sin dejar de recibir tráfico

Big Data - SuperWebAnalytics.com



5 - Fallas en nodos de la base:

- Al aumentar la cantidad de nodos, se hace frecuente que algún disco deje de funcionar.
- Hay que hacer **fault tolerant** a la base de datos:
 - Es necesario empezar a mantener registro de qué nodos están **online** y cuáles no para los workers.
 - Es necesario mantener **réplicas pasivas** de los nodos para poder seguir sirviendo consultas.

Big Data - SuperWebAnalytics.com



6 - Fallas en el código (bugs):

- Al introducir un bug en el código de los workers, la información en nuestra base de datos (source of truth) se vuelve corrupta:
 - Los backups no alcanzan, porque implican perder información.
 - No se puede saber qué información está corrupta y cómo.
 - Hay que buscar la *human fault tolerance*

Big Data - SuperWebAnalytics.com



First principles:

¿Cuál es el **objetivo de los sistemas basados en datos**?

- **Contestar preguntas** basándose **en información recolectada** hasta ese momento.
 - ¿Cuál es el **nombre de este usuario**?
 - ¿**Cuántos amigos tiene** este usuario?
 - ¿Cuál es mi **balance actual**?
 - ¿Cuáles fueron **las transacciones recientes en mi cuenta**?

Big Data - SuperWebAnalytics.com



Otra característica crucial es que no todos los bits de información son iguales. Cierta información es derivada de otras piezas de información.

Si se busca el origen desde donde fue generada una información, finalmente se llegará a un fuente que no es derivable de ninguna otra fuente.

➤ **Data:** información que es verdadera simplemente por existir.

Data será, entonces, información especial desde la cual toda otra información puede ser derivada.

Big Data - SuperWebAnalytics.com



Si un sistema de datos responde preguntas mirando datos del pasado, entonces el sistema de datos más general responde preguntas mirando el conjunto de datos completo. Así, la definición más general que podemos dar para un sistema de datos es la siguiente:

`query = function(all data)`

Big Data - Requerimientos no funcionales



¿Qué **atributos de calidad son deseables?**

- Tolerancia a fallos
- Escalabilidad
- Extensibilidad
- Mantenibilidad
- Baja latencia
- Soporte para consultas específicas

Soluciones de Data Storage

Data Storage



— — — Data warehousing / Data Lake

- ❖ Arquitecturas con fines analíticos y de reporting
 - Business Intelligence / Decision Making
- ❖ No transaccionales (baja o nula mutabilidad)
- ❖ Repositorios centrales que unifican muchas fuentes de datos diversas
- ❖ Orientados a muy grandes volúmenes de datos
- ❖ Pensados para escalabilidad

Data Storage - Data warehousing

— — —



❖ Datos fuertemente estructurados

- Estructura impuesta a los datos de diversas fuentes (ETL)
- Estructura orientada a los casos de análisis y reporting necesarios
- Estrategia de schema-on-write

- **Schema on write:** Necesita que las estructuras estén definidas previa a la carga. Los datos son validados contra estas estructuras.
- **Schema on read:** En este caso, la estructura de los objetos de BBDD no está definida previa a la carga y, entonces, no hay validación contra estructura. La estructura de BBDD se define en la lectura, con la flexibilidad de poder cambiar la estructura en función de los datos a obtener en cada tipo de lectura.

Ventajas e inconvenientes de cada una de las arquitecturas:

•Schema on write:

- Al realizar la validación de datos contra la estructura en tiempo de carga, son los volcados de datos los que son penalizados, mientras que las lecturas son más rápidas. Al existir una estructura predefinida, hay una única forma de visualizar los datos. Cualquier cambio en el esquema (nueva columna), obliga a reprocesar. Al tener estructuras predefinidas la relación entre entidades es sencilla de obtener y los datos pueden quedar documentados en la estructura. Por otra parte, bajo este esquema resulta complicado mapear datos no estructurados y resulta muy costoso mantener el nivel de detalle más atómico para volúmenes de datos altos.

Ventajas e inconvenientes de cada una de las arquitecturas:

•Schema on read:

- Al no realizar validaciones en tiempo de carga, los volcados son más rápidos: la penalización en tiempos está en la lectura cuando tenemos una estructura. Al no haber un esquema predefinido, tenemos la flexibilidad de poder leer los datos según la vista que nos interese en cada tipo de consulta; por el contrario, no existe una documentación clara de los datos. Este esquema es muy apropiado para datos no estructurados y permite guardar el nivel de detalle más atómico, incluso en volúmenes de datos altos, aunque necesita una capacidad de proceso muy alta.

El uso de uno u otro esquema va a depender mucho del tipo de datos a almacenar/consultar y de la capacidad de proceso con la que contamos.

Data Storage - Data warehousing

— — —



❖ Normalización de datos

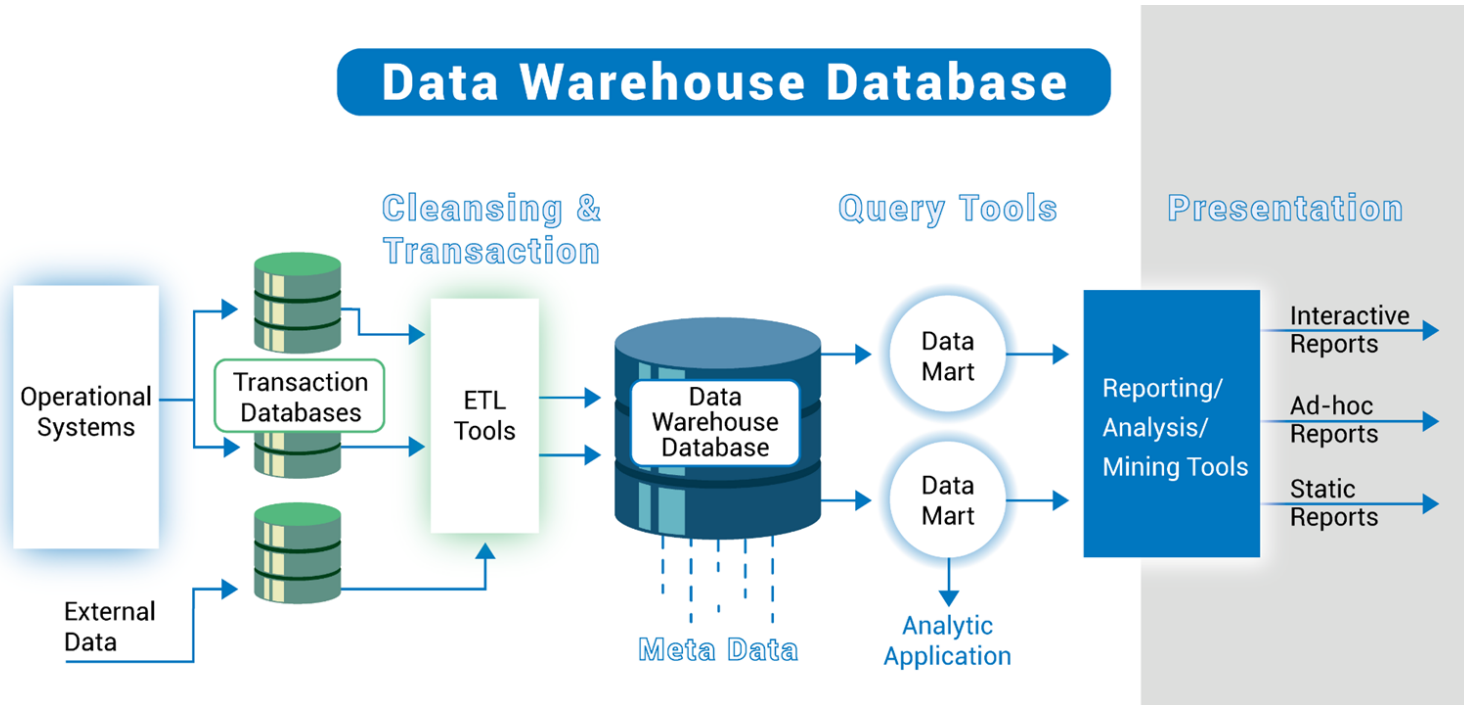
- Evitar redundancia y aumentar la integridad

❖ Data cleaning

- Imponer atributos de calidad de los datos
- Descartar lo que no se encuadre

❖ Data governance y seguridad robustas

Data Storage - Data warehousing



Data Storage - Data warehousing



— — —

Componentes principales:

- ❖ El almacenamiento (RDBMS para distintas capas)
- ❖ Pipelines de ETL (Extract, Transform, Load)
- ❖ Capa de metadata (Para describir los datos almacenados)
- ❖ Data marts (capa de consulta, casos específicos)
- ❖ Capa de presentación (análisis y reporting)

Data Storage - Data warehousing



Soluciones cloud based

- ❖ BigQuery: <https://cloud.google.com/bigquery>
- ❖ RedShift: <https://aws.amazon.com/redshift/>
- ❖ Snowflake: <https://www.snowflake.com/en/data-cloud/platform/>

También pueden encontrarse soluciones para implementar *on premises*

Data Storage - Data lake

— — —



- ❖ Datos crudos de las distintas fuentes
 - No se impone estructura al ingestar (ELT)
 - No se descartan ni limpian datos
 - Datos estructurados (JSON, XML) y no estructurados (imágenes)
 - Estrategia schema-on-read
- ❖ Inmutabilidad de los datos
- ❖ Más dificultades para lograr data governance y seguridad
- ❖ Análisis y reporting, pero también otros casos como Machine Learning

Pipelines de ingestión

ETL: Extract, Transform, Load



ETL se refiere a un proceso por el cual la información fluye entre 2 sistemas mediante 3 etapas bien definidas:

- **Extract:** la información es consultada y extraída del sistema de origen.
- **Transform:** la información es limpiada, normalizada y procesada para adaptarse al sistema de destino.
- **Load:** la información ya transformada es almacenada en el sistema de destino.

ETL: Extract, Transform, Load



ETL no es propio de ninguna arquitectura en particular, sino que es una actividad que aparece en distintos puntos. Por ejemplo:

- En un **data warehouse** se aplica ETL en la etapa de **staging/integración**.
- Se aplica ETL al momento de leer los datos de un **data lake** y utilizarlos para algún fin específico.
- Se aplican procesos ETL al **migrar información**, por ejemplo si la empresa migra de un sistema contable a otro.

ETL: Extract, Transform, Load



— — —
Ejemplo:

- Se tiene un sistema de facturación que funciona en cada sucursal y almacena la información relativa a sus ventas.
- En un data warehouse central se almacena por cada sucursal el monto vendido por mes.
- Un proceso **ETL** se ejecuta al fin de mes en cada una de las sucursales:
 - **E**: se obtienen las facturas del mes desde el sistema de facturación.
 - **T**: se calcula el total de todas las facturas del mes.
 - **L**: se almacena el total del mes en el data warehouse.

ETL: Extract, Transform, Load

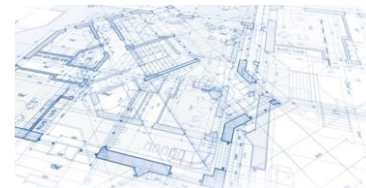


— — —

Los procesos ETL son muy habituales en el **software corporativo** y es por eso que existen diversas soluciones “enlatadas”:

- IBM DataStage
- Microsoft SSIS
- Adeptia ETL Suite
- Oracle Data Integrator
- CloverETL (Open Source)

ELT: Extract, Load, Transform



- Las técnicas de **ELT** son muy similares a las de **ETL**.
- El objetivo es el mismo: **mover información** de un sistema a otro.
- En el caso de ELT los pasos son ligeramente distintos
 - **Extract**: la información es consultada y extraída del sistema de origen.
 - **Load**: la información cruda es almacenada en el sistema de destino.
 - **Transform**: es el mismo sistema de destino quien se encarga de limpiar, normalizar y transformar la información cruda.

ELT: Extract, Load, Transform



— — —

Mismo ejemplo: sistema de facturación, warehouse mensual.

El proceso **ELT** se ejecuta al fin de mes en cada una de las sucursales:

- **E**: se obtienen las facturas del mes desde el sistema de facturación.
- **L**: se almacenan las facturas del mes en el data warehouse central.
- **T**: la BD del warehouse calcula el total de todas las facturas del mes para cada sucursal.